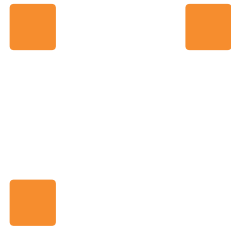


Linear Models

For classification and regression.

Simple models, easy to understand and fast to train.



Outline

- What is a linear model?
- For regression: linear regression
- For classification: logistic regression
- Non linearly separable data

An example: estimating housing prices

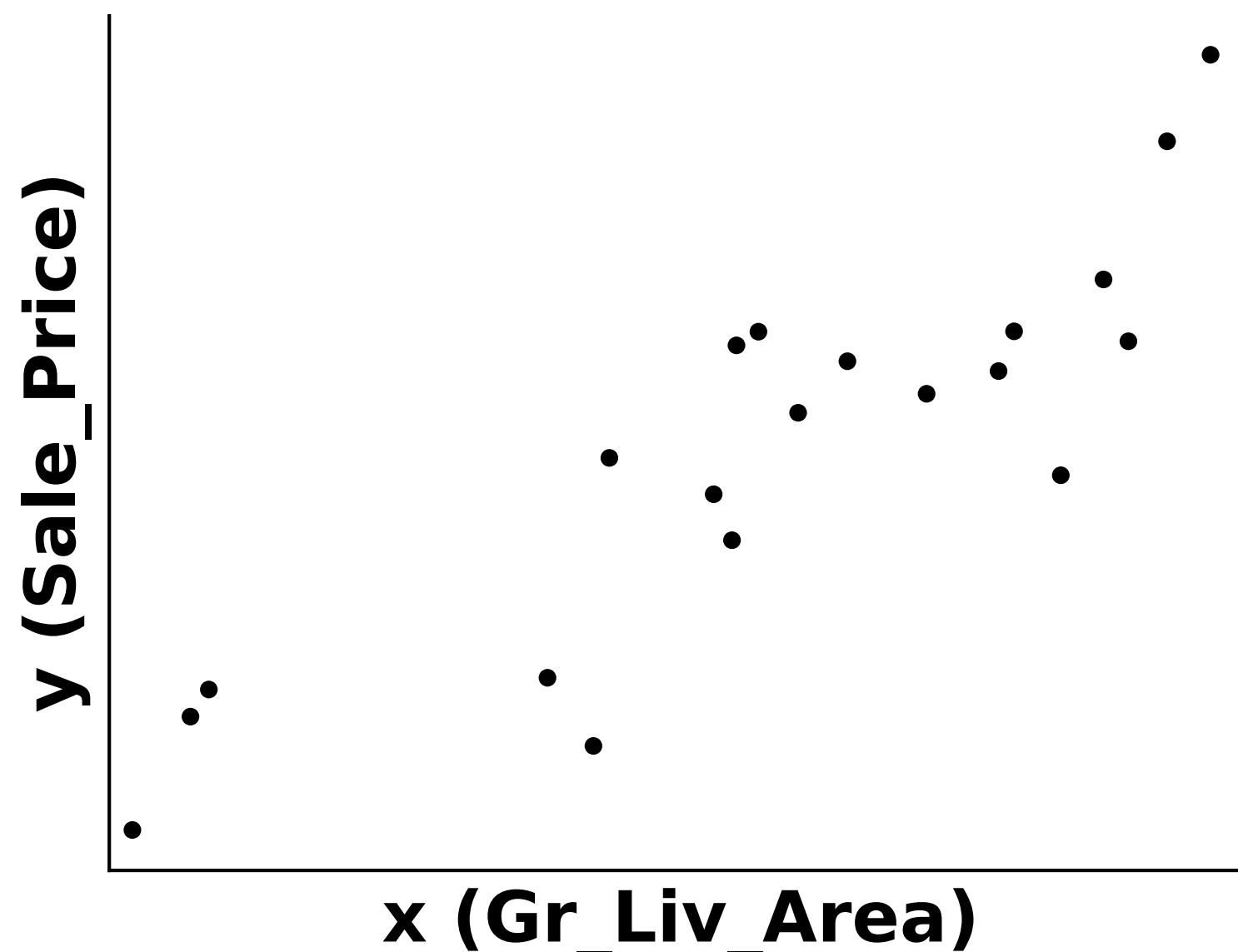
Gr_Liv_Area	Year_Built	Full_Bath	Sale_Price
1656	1960	1	215.0
896	1961	1	105.0
1329	1958	1	172.0
2110	1968	2	244.0
1629	1997	2	189.9

Linear approximation
of Sale_Price:

$$\begin{aligned}\text{Sale_Price} = & 0.1 * \text{Gr_Liv_Area} \\ & + 1.1 * \text{Year_Built} \\ & - 8.9 * \text{Full_Bath} \\ & - 2200.0\end{aligned}$$

Linear regression

Predict the value of the target y given some observation x

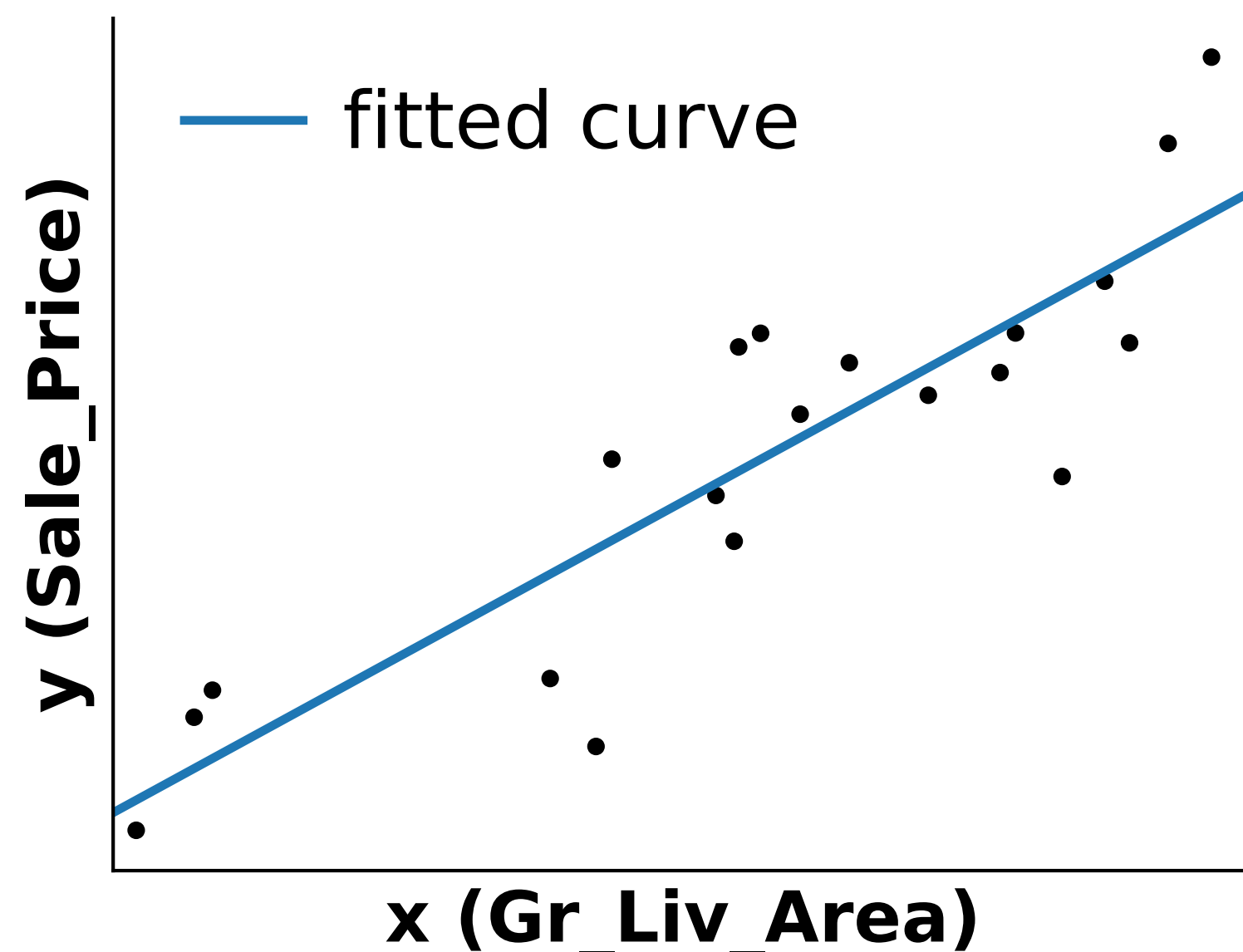


Linear approximation
of Sale_Price:

$$\text{Sale_Price} = 0.1 * \text{Gr_Liv_Area} - 2200.0$$

Linear regression

Predict the value of the target y given some observation x

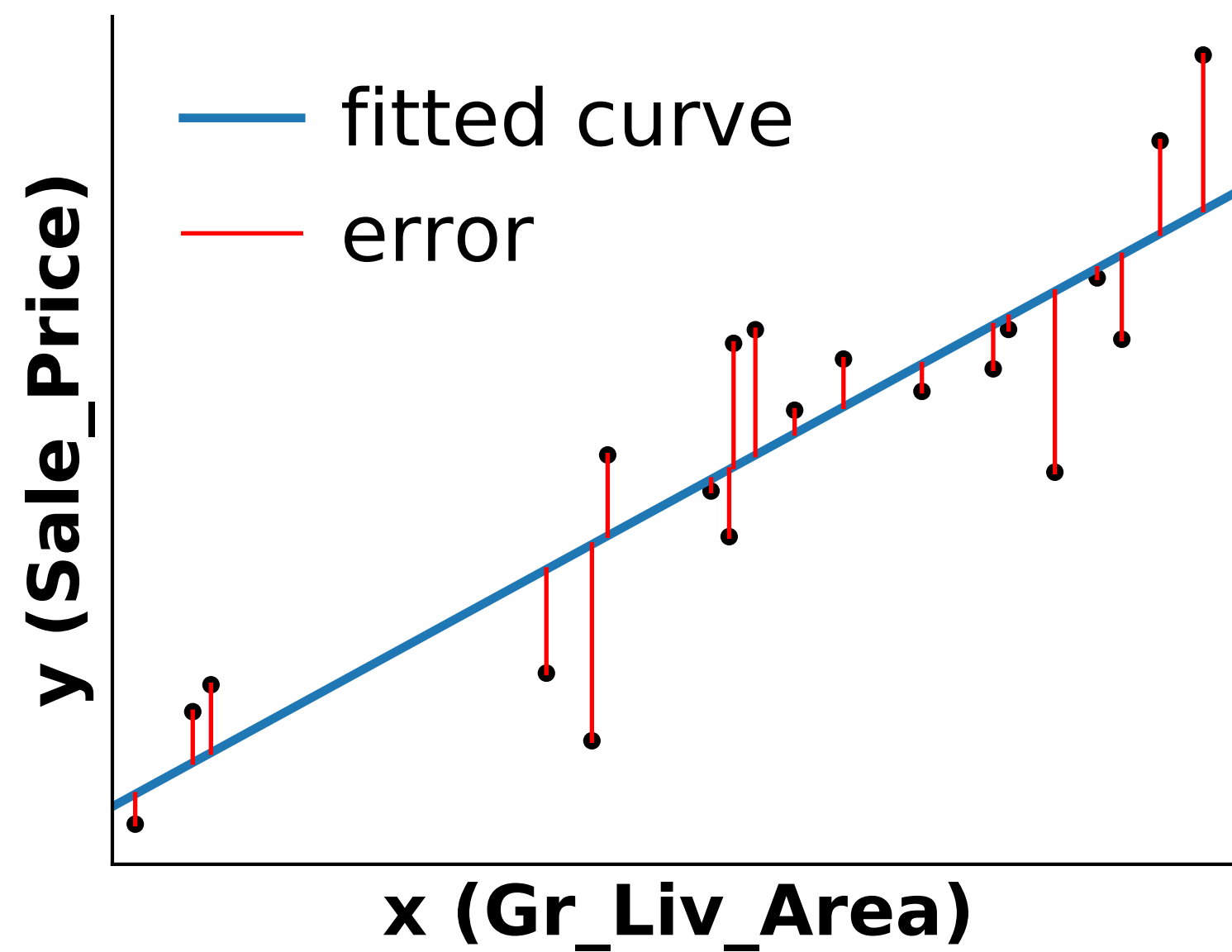


```
from sklearn.linear_model import LinearRegression

linear_regression = LinearRegression()
linear_regression.fit(X, y)
```


Linear regression

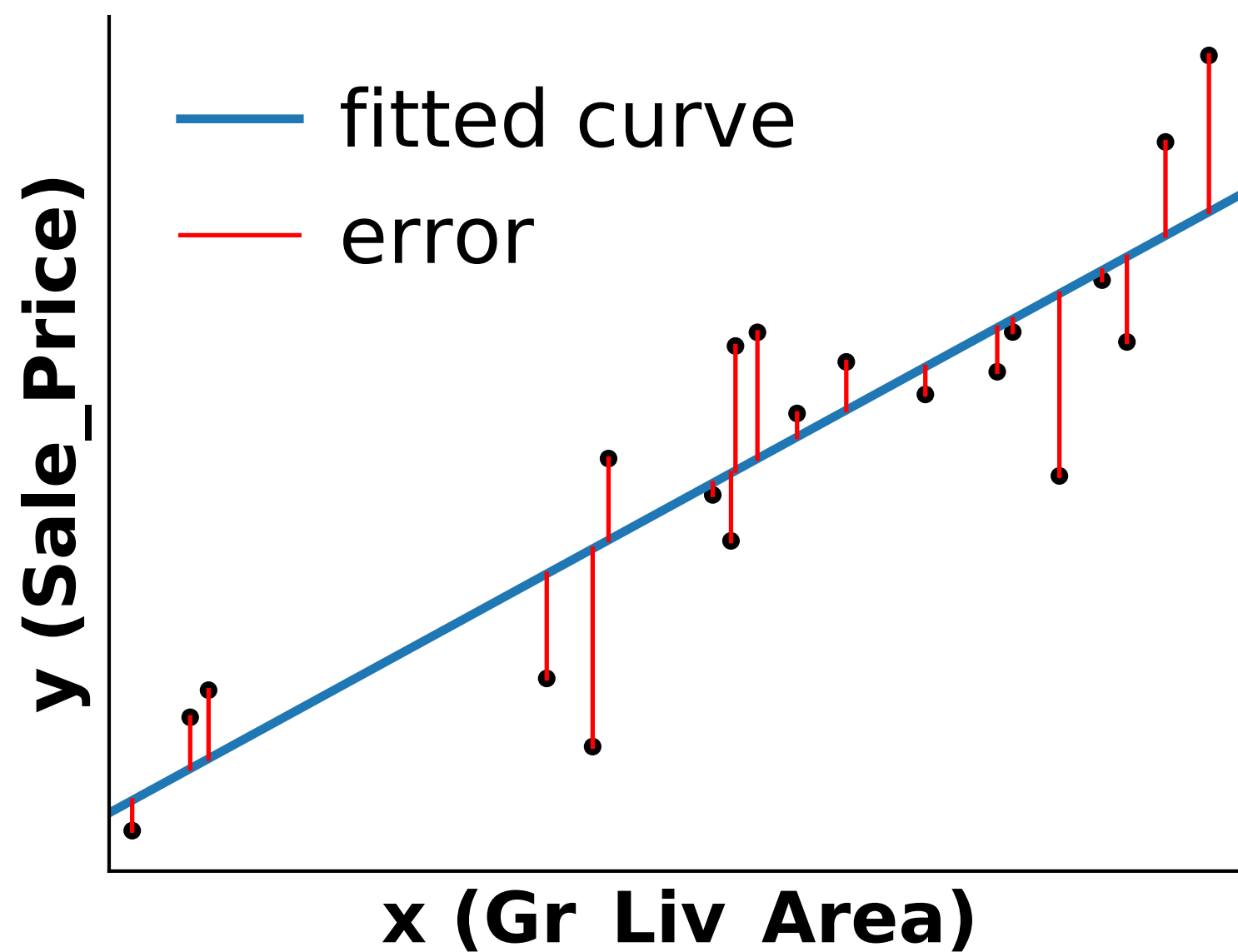
The slope minimizes the distance between predictions and data points



```
from sklearn.linear_model import LinearRegression  
  
linear_regression = LinearRegression()  
linear_regression.fit(X, y)
```

Linear regression

The slope minimizes the distance between predictions and data points



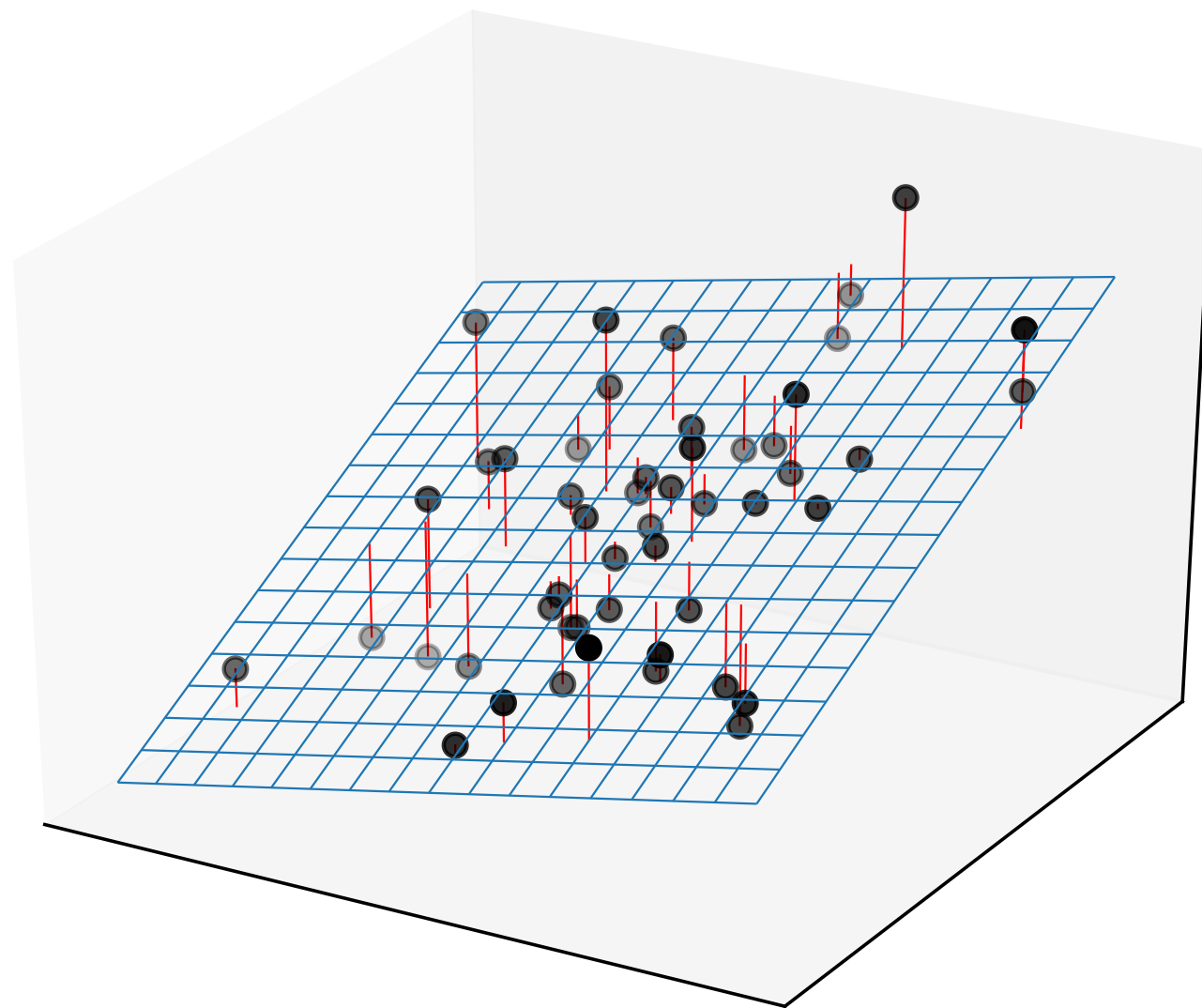
```
from sklearn.linear_model import LinearRegression
```

```
linear_regression = LinearRegression()
linear_regression.fit(X, y)
```

```
import numpy as np
```

```
y_pred = linear_regression.predict(X)
squared_error = np.sum((y - y_pred) ** 2)
```

Linear regression with several variables



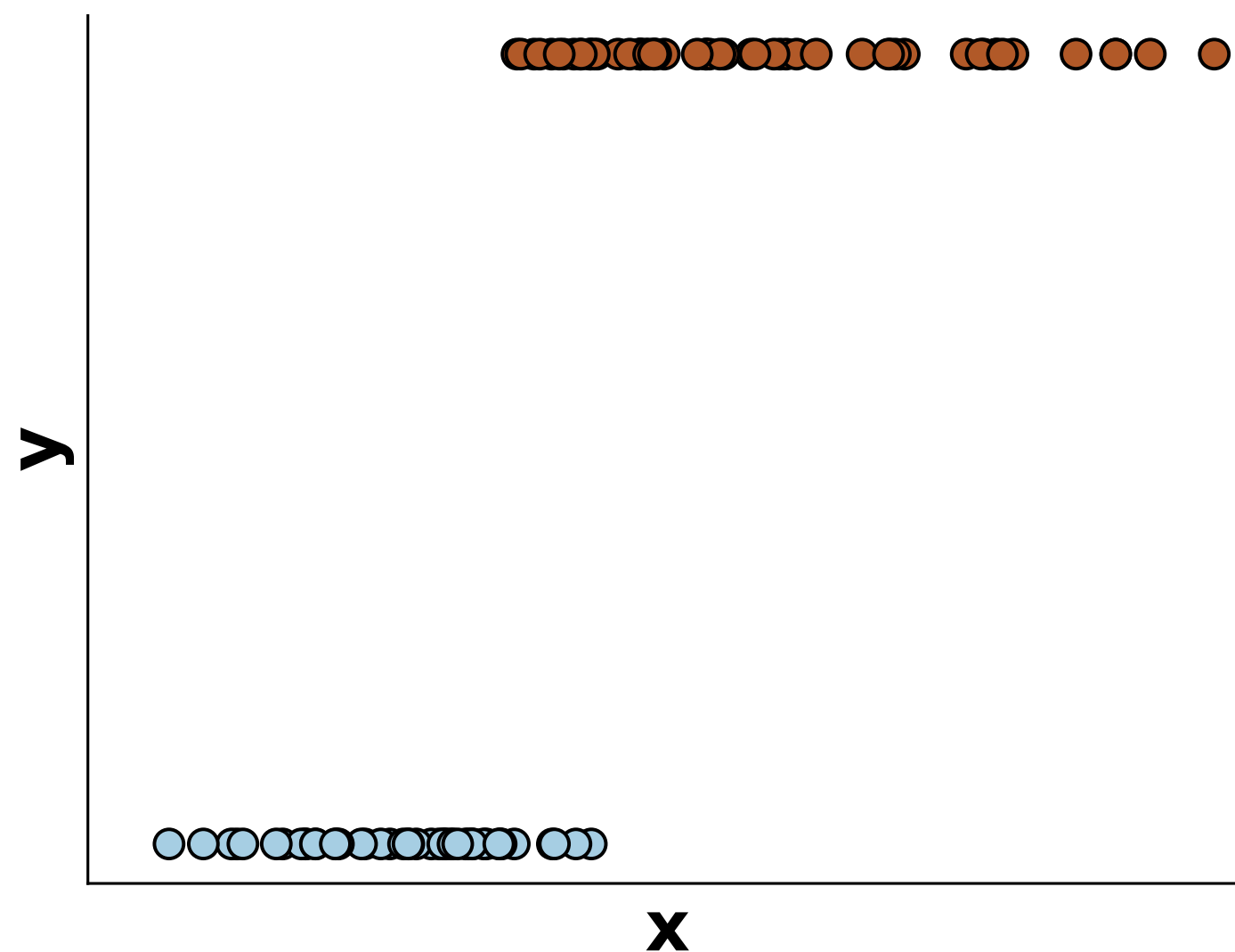
Extend mental picture
to more dimensions.

For instance in 2D:

$$\begin{aligned}\text{Sale_Price} = & 0.1 * \text{Gr_Liv_Area} \\ & + 1.1 * \text{Year_Built} \\ & - 2209.0\end{aligned}$$

For classification: logistic regression

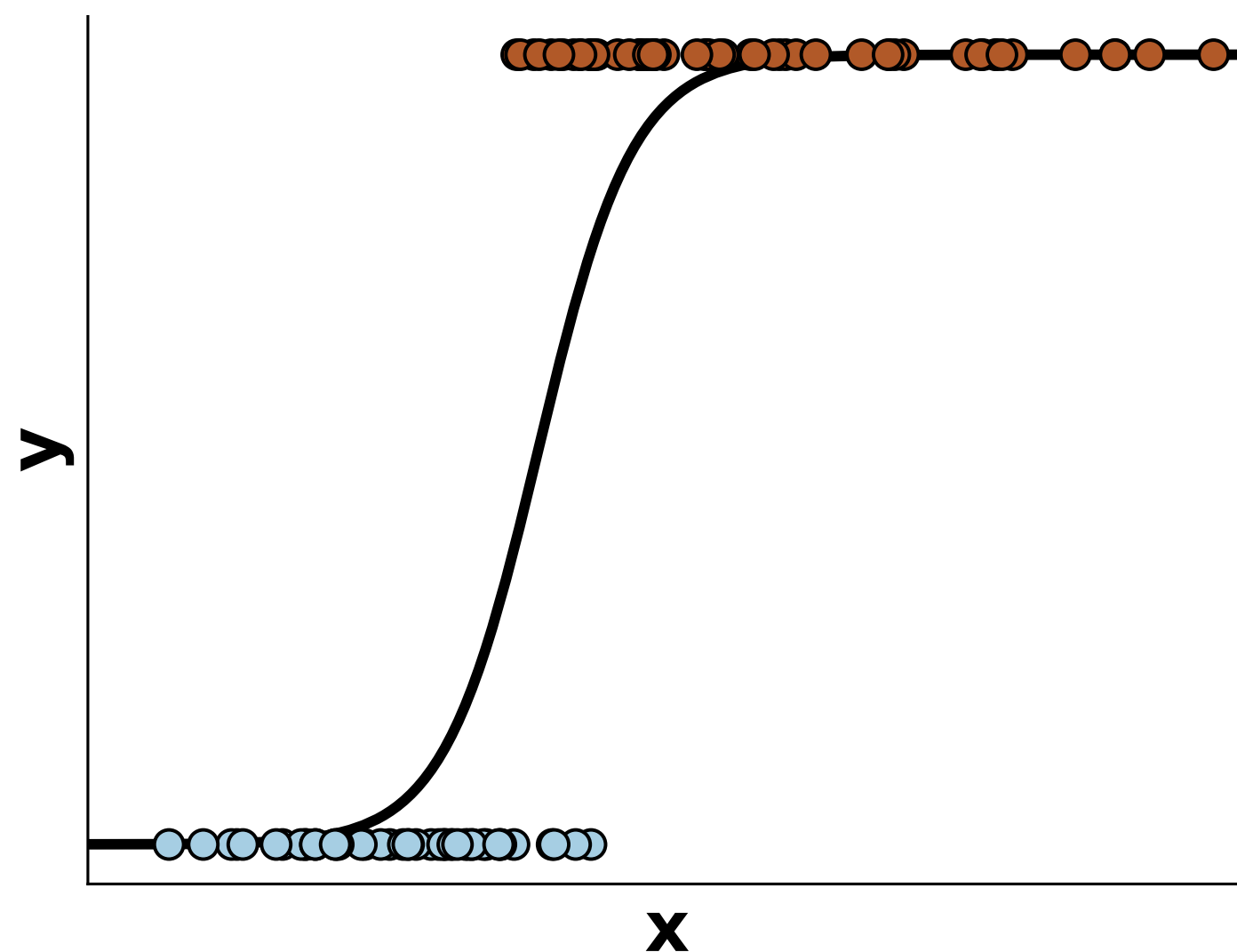
For classification, we use a logistic regression model:
y is either 0 (blue) or 1 (red)



```
from sklearn.linear_model import LogisticRegression  
  
log_reg = LogisticRegression()
```

For classification: logistic regression

For classification, we use a logistic regression model:
y is either 0 (blue) or 1 (red)

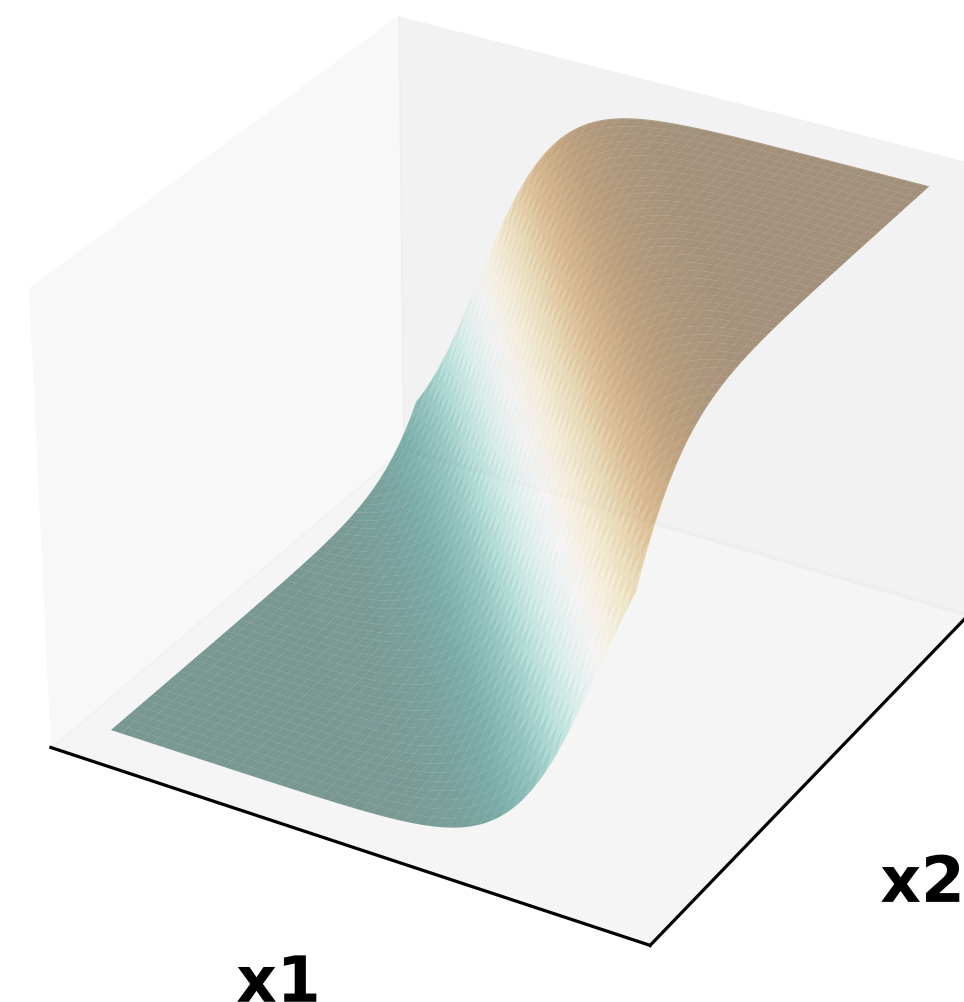
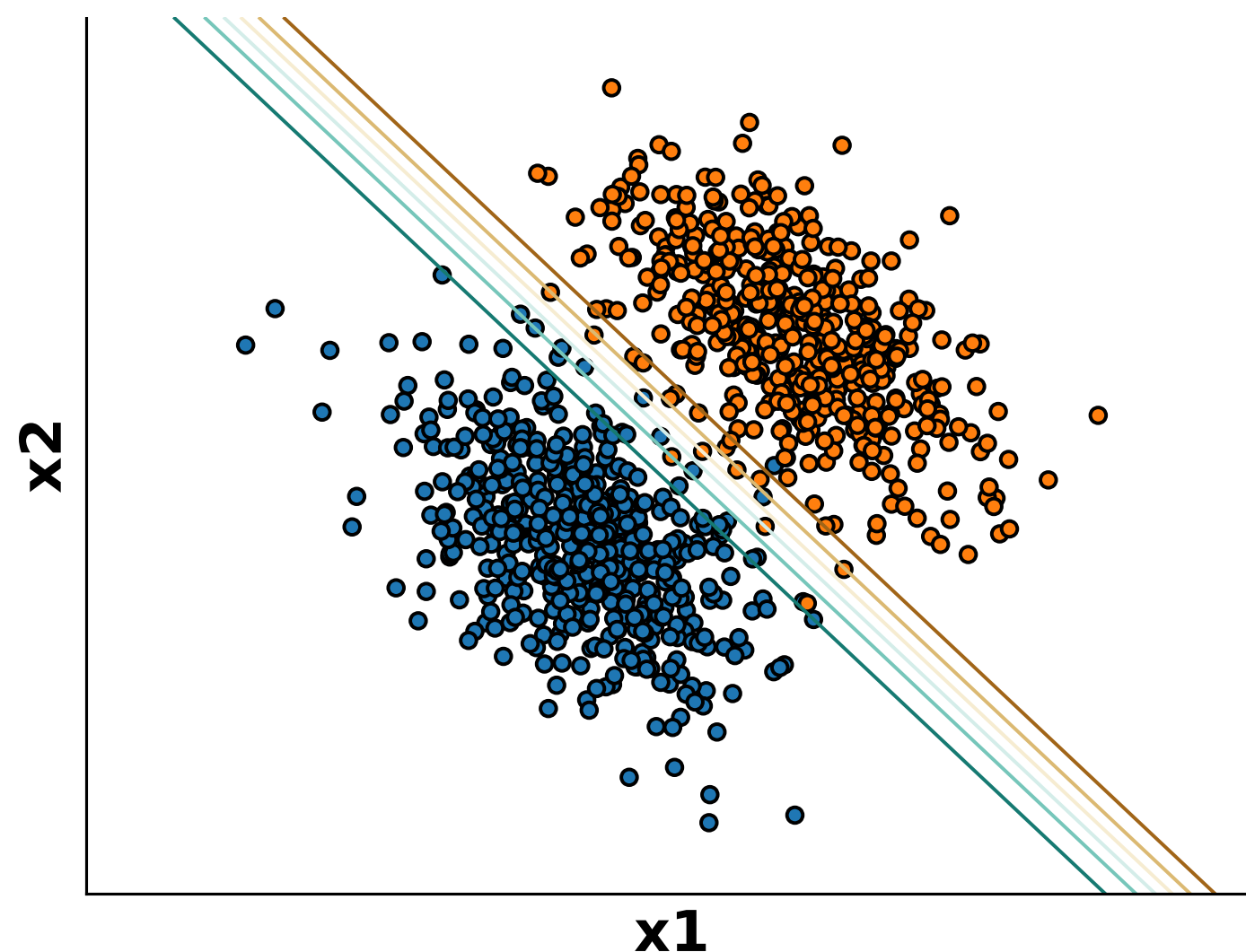


```
from sklearn.linear_model import LogisticRegression

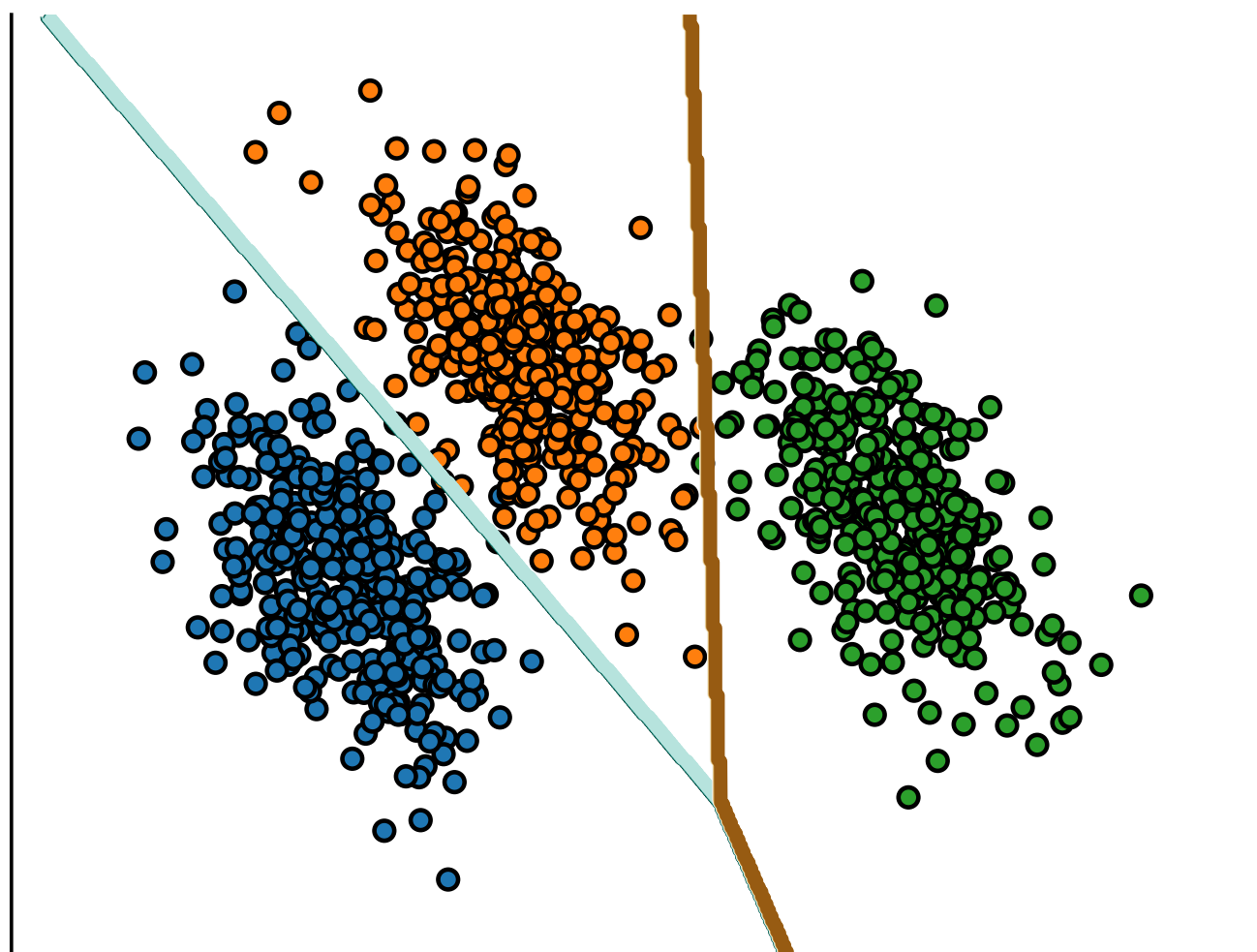
log_reg = LogisticRegression()
log_reg.fit(X, y)
```

Logistic regression in 2 dimensions

X is 2-dimensional, y is represented by the color



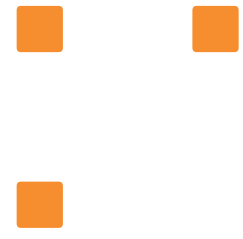
Logistic regression & multiclass classification



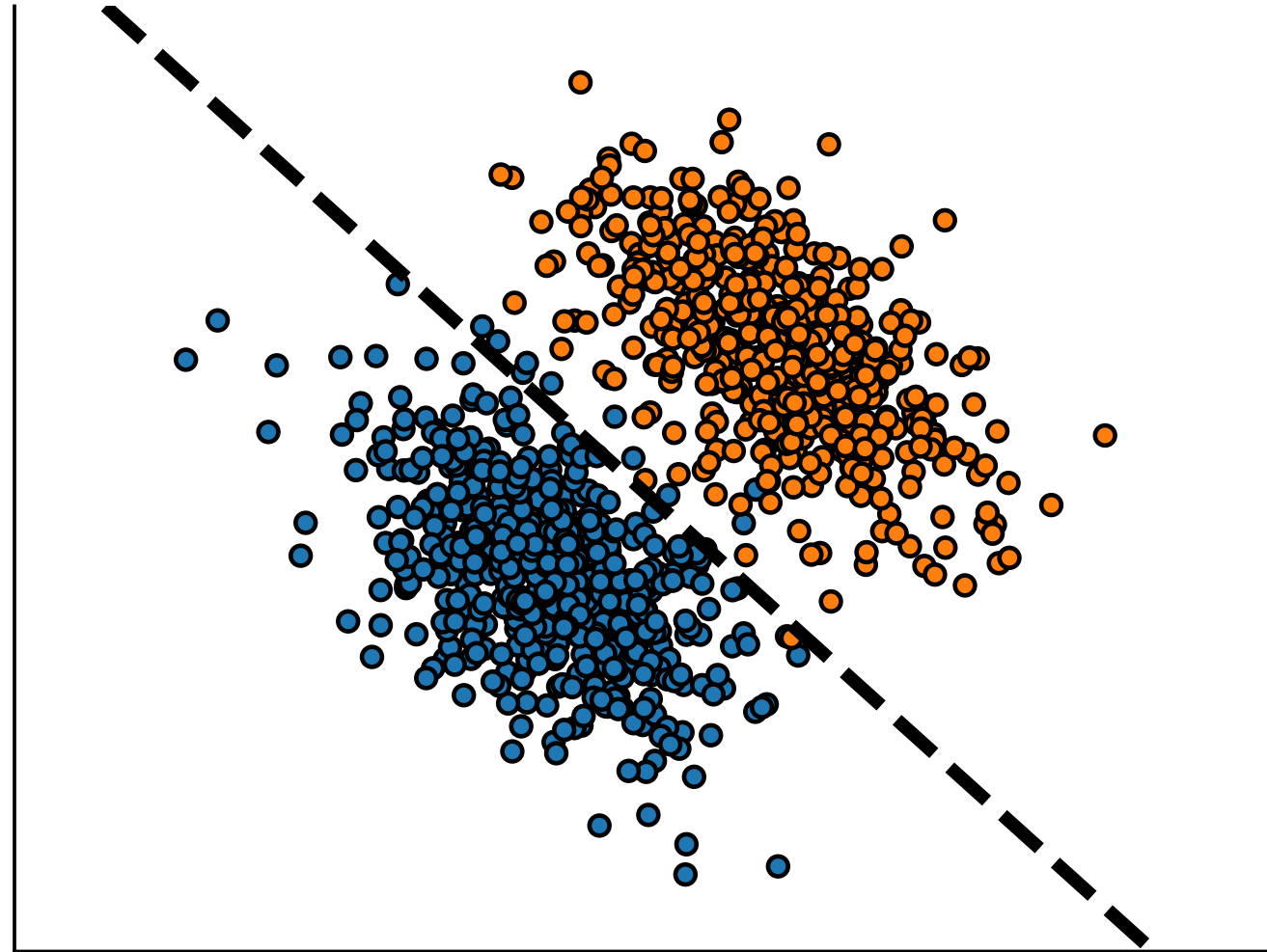
- $y \in \{0, 1, 2\}$
- $y \in \{\text{blue, orange, green}\}$

For a given input x :

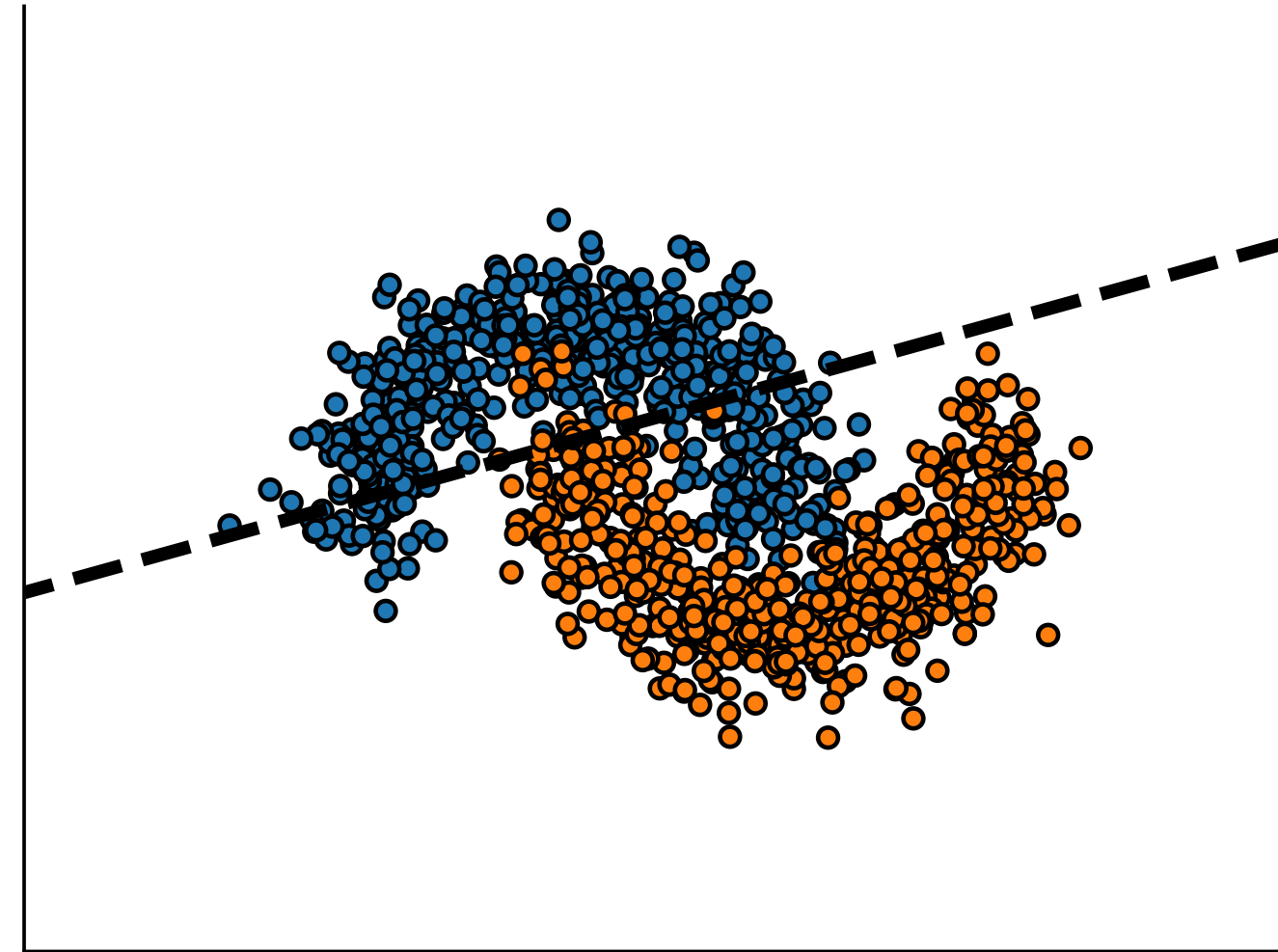
- predict one probability per class
- probabilities sum to 1



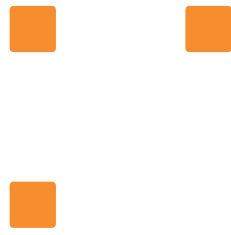
Linear models are not suited to all data



Almost linearly separable

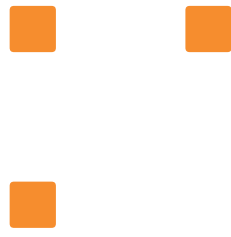


Not linearly separable
→ underfitting



Feature engineering

- data can become linearly separable in a new feature space with more dimensions
 - by collecting data from another feature
 - by creating new features derived from the existing ones



Take home messages on linear models

- Simple and fast baselines for:
 - regression: linear regression
 - classification: logistic regression
- Can underfit if too few features
→ engineering new features can help!
- Hard to beat when `n_features` is large
- May still overfit